Python for AI-Driven Automation and Business Data Science

Companion slides to the 11-notebook course

Course materials

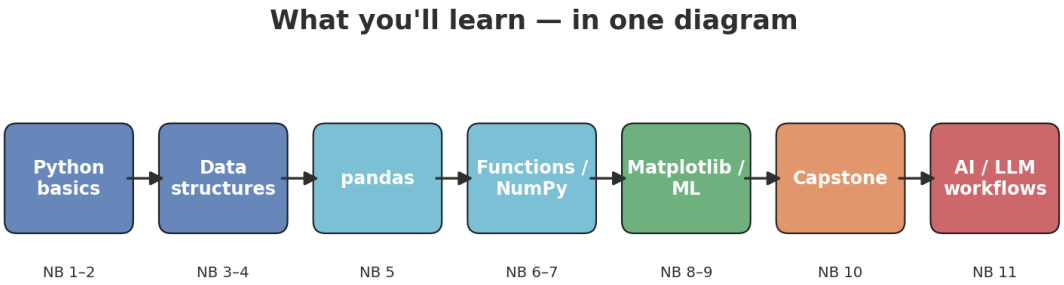
Contents

1. Overview
2. Python
3. HTTP & SQL
4. Data Handling
5. NumPy
6. Visualisation
7. Time Series

Outline

1. Overview
   - The course in one diagram
   - Why this course
2. Python
3. HTTP & SQL
4. Data Handling
5. NumPy
6. Visualisation

What you will learn



Each step builds on the one before. The last step is what makes the whole thing modern.

Why this course exists

The 2015 way of teaching Python

Print statements, fizz-buzz, a quick tour of pandas, maybe a linear regression at the end.

What working with data actually looks like in 2025

- Calling APIs, parsing JSON, retrying on failure.
- Cleaning messy text, building dashboards, training small models.
- Calling an LLM as a function – and getting structured output back.

This course teaches Python through the second lens from the very first cell.

Three skills the modern data role asks for

1. **Solid Python fluency** – reading and writing code without friction.
2. **Working with data professionally** – cleaning, exploring, modelling, visualising.
3. **AI-driven automation** – using LLMs, APIs, and small scripts to replace repetitive analytical work.

All three are exercised across the 11 notebooks.

Course at a glance – 19 notebooks, four arcs

NB	Topic	What you build
Foundations		
1	Python basics	KPI snapshot for an AI support bot
2	Control structures	Ticket-triage rules engine
3	Lists & sequences	Latency-log analysis
4	Dictionaries / JSON	Defensive API-response parser
5	Pandas preview	LLM-call log analysis
6	Functions & modules	Reusable cost / cleaning toolkit
Real-world I/O		
12	APIs & HTTP	Weather-data ETL against a live API
13	SQL fundamentals	SQL-driven channel report
Data science & ML		
7	NumPy fundamentals	A/B-test of two LLM providers
8	Matplotlib	2×2 AI-ops dashboard
14	Time series & forecasting	3-month forecast of automation rate
9	scikit-learn	Customer-churn prediction
10	**Capstone**	AI Support-Bot Analytics
AI engineering		
11	AI workflows	Inbox-triage with the MockLLM

Outline

1 Overview

2 Python
- Variables and types
- Arithmetic for AI workloads
- Decisions and loops
- Functions

3 HTTP & SQL

4 Data Handling

5 NumPy

Variables – labels on values

From an AI-automation project

```
model_name = "gpt-4o-mini" # which model
price_per_1k = 0.0006 # USD per 1K input tokens
monthly_tokens = 14_500_000 # expected volume
is_production = True
```

- Five core types you meet daily: `int`, `float`, `str`, `bool`, `None`.
- `snake_case` names that mean something to a stranger.
- Underscores in numbers (`14_500_000`) just for readability.

Strings arrive from APIs – and they look like numbers

The most common parsing bug

`"25" + 5` raises `TypeError`: you cannot add a string to an int.

```
raw_tokens = "1024" # comes back as a string
tokens = int(raw_tokens) # convert before doing maths

int(3.9) # 3 -- truncates toward zero, does NOT round
round(3.9) # 4
```

Convert with `int()`, `float()`, `str()`, `bool()` – the bread and butter of cleaning real data.

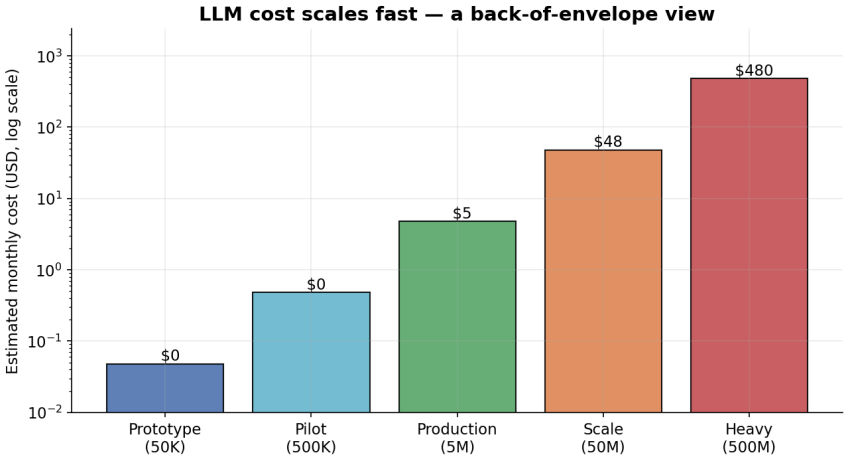
Costing a batch of LLM calls in 4 lines

```
n_tickets = 350
tokens_per_call = 600
price_per_1k_tok = 0.0006

total_cost = n_tickets * tokens_per_call / 1000 * price_per_1k_tok
print(f"Total cost: ${total_cost:.4f}") # Total cost: $0.1260
```

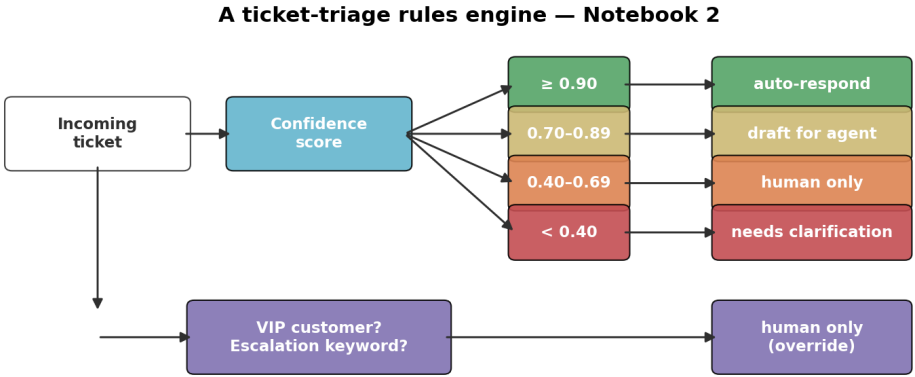
The pattern that scales

Inputs at the top → *derived values* → *readable output*. Edit one input, re-run, done.

Token cost scales fast



A rules engine for ticket triage



If the override branch fires, it wins regardless of confidence.

The same engine in Python

```python
def route_ticket(confidence, is_vip, message):
    text = message.lower()
    if any(w in text for w in ("urgent", "escalate", "refund")):
        return "human-only"
    if is_vip and confidence < 0.8:
        return "human-only"
    if confidence >= 0.90: return "auto-respond"
    if confidence >= 0.70: return "draft-for-agent"
    if confidence >= 0.40: return "human-only"
    return "needs-clarification"
```

Branch order matters

Most restrictive condition first. Otherwise enterprise customers fall into the SMB tier.

Retry-with-backoff – the `while` pattern

```
attempt, success = 0, False
while attempt < max_attempts and not success:
    attempt += 1
    success = call_api()
    if not success:
        wait = 0.5 * 2 ** (attempt - 1) # exponential backoff
        time.sleep(wait)
```

Two stop conditions joined by `and`

Keep trying while I haven't succeeded ***and*** *I haven't burned through my budget.* The safest shape for any retry loop.

Wrap reusable logic behind a name

```python
def call_cost(tokens_in, tokens_out,
              price_in_per_1k=0.0006,
              price_out_per_1k=0.0024) -> float:
    """USD cost of one LLM call from token usage."""
    return (tokens_in / 1000 * price_in_per_1k +
            tokens_out / 1000 * price_out_per_1k)

call_cost(800, 200) # $0.000960
call_cost(800, 200, 0.0010) # $0.001280 -- override one default
```

Four wins you get for free

A meaningful *name*, *testability*, *reusability*, *one place to change* when the price moves.

Outline

1 Overview

2 Python

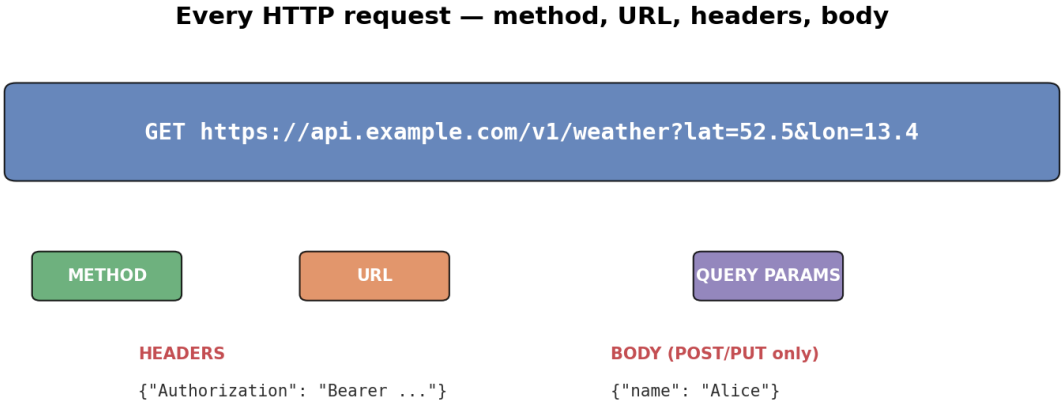
3 HTTP & SQL
- HTTP for data fetching
- SQL fundamentals

4 Data Handling

5 NumPy

6 Visualisation

Every HTTP request, in one picture



The five rules that save you in production

1. Always pass `timeout=...` (or your code hangs forever)
2. Call `r.raise_for_status()` *before* `r.json()`
3. Retry on 429 and 5xx; **not** on 400/401/404
4. Exponential backoff: `0.5s` $\rightarrow$ `1s` $\rightarrow$ `2s` $\rightarrow$ `4s`
5. API keys in environment variables, never in code

The defensive shape

```python
try:
    r = requests.get(url, params=p, timeout=8)
    r.raise_for_status()
    return r.json()
except requests.exceptions.RequestException as e:
    log.warning(f"fetch failed: {e}")
    return None
```

SQL vs pandas – same answer, two languages

Same analysis, two languages — pick the one that reads better

SQL

```
SELECT channel,
       AVG(automation_rate) AS mean_auto,
       SUM(tickets_total)   AS total
FROM     support_ops
WHERE    automation_rate > 0.5
GROUP BY channel
ORDER BY total DESC
```

pandas

```
(support_ops
   .query('automation_rate > 0.5')
   .groupby('channel')
   .agg(mean_auto=('automation_rate','mean'),
        total=('tickets_total','sum'))
   .sort_values('total', ascending=False))
```

When SQL beats pandas (and vice versa)

Reach for SQL when ...

- The data lives in a database – don't move it before you have to.
- Joining several large tables on keys.
- Filtering / aggregating before bringing data into Python.
- The query needs to be shared with non-Python colleagues.

Reach for pandas when ...

- The data is already a DataFrame.
- Reshaping (pivot, melt, multi-index).
- You need to plug into matplotlib / scikit-learn directly.

Pro pattern: SQL for the heavy lift, pandas for the last mile.

Outline

1 Overview

2 Python

3 HTTP & SQL

4 Data Handling
- The three core shapes
- Pandas preview

5 NumPy

6 Visualisation

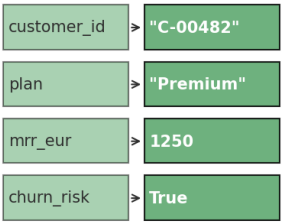
List vs dict vs DataFrame

list — ordered values

latencies (ms)

lookup by position

dict — key → value



lookup by name

DataFrame — table of records

id	model	cost	lat
001	gpt	0.0006	1.8s
002	gpt	0.0009	2.1s
003	cld	0.0008	1.6s
004	gpt	0.0014	3.2s

select / filter / group

Lists – ordered, mutable, sliceable

```
latencies_ms = [180, 195, 210, 178, 232, 169, 204]

latencies_ms[0] # 180 (first)
latencies_ms[-1] # 204 (last)
latencies_ms[-3:] # [169, 204] wait, that's only 2 -- need [-3:]
latencies_ms[::2] # every other element
```

The single most important pandas-style syntax – and it works on plain Python lists, NumPy arrays, and pandas Series unchanged.

Dicts – the JSON / API / LLM shape

```
api_response = {
    "request_id": "req_9472193",
    "model": "gpt-4o-mini",
    "usage": {"input_tokens": 482,
                "output_tokens": 121,
                "total_tokens": 603},
    "choices": [{"message": {"role": "assistant",
                             "content": "Refunds take 5 business days."}}],
}

reply = api_response["choices"][0]["message"]["content"]
```

Same shape as every web API and every LLM provider's chat format.

Defensive parsing – the production habit

```python
def safe_get(d, path):
    """Walk a nested dict/list. Return None on missing/wrong type."""
    cur = d
    for step in path:
        try:
            cur = cur[step]
        except (KeyError, IndexError, TypeError):
            return None
    return cur

safe_get(resp, ["choices", 0, "message", "content"]) # works
safe_get(resp, ["choices", 5, "message"]) # None, no crash
```

Rule of thumb

Whenever you read data you don't fully control, write the access function *defensively once* and use it everywhere.

A DataFrame is a table you can program against

```python
import pandas as pd

df = pd.DataFrame({
    "request_id": ["req_001", "req_002", "req_003"],
    "model": ["gpt-4o-mini", "claude-haiku", "gpt-4o-mini"],
    "tokens_in": [482, 612, 1180],
    "tokens_out": [121, 158, 205],
})

df.shape # (3, 4)
df["tokens_in"].mean() # average input tokens
df[df["tokens_in"] > 500] # filter rows
df.groupby("model")["tokens_in"].sum()
```

The five commands you always run first

Inspect any new dataset

- `df.head()` – look at the first few rows
- `df.shape` – rows $\times$ columns
- `df.dtypes` – what's a string vs a number
- `df.describe()` – per-column summary statistics
- `df.info()` – types + missing-value counts

Skipping this step is the most common rookie mistake.

Filter, derive, group – the pandas trio

```python
# Filter: boolean mask
slow = df[df["latency_ms"] > 2000]

# Derive: vectorised arithmetic, no loop
df["cost_usd"] = (df["tokens_in"] / 1000 * 0.0006 +
                  df["tokens_out"] / 1000 * 0.0024)

# Group: split -> apply -> combine
df.groupby("model").agg(
    n_calls =("request_id", "count"),
    mean_cost =("cost_usd", "mean"),
)
```

These three operations cover 80% of analytical work.

Outline

1 Overview

2 Python

3 HTTP & SQL

4 Data Handling

5 NumPy
- Why NumPy
- Vectorised arithmetic

6 Visualisation

NumPy is the engine under everything

Why it matters for AI work

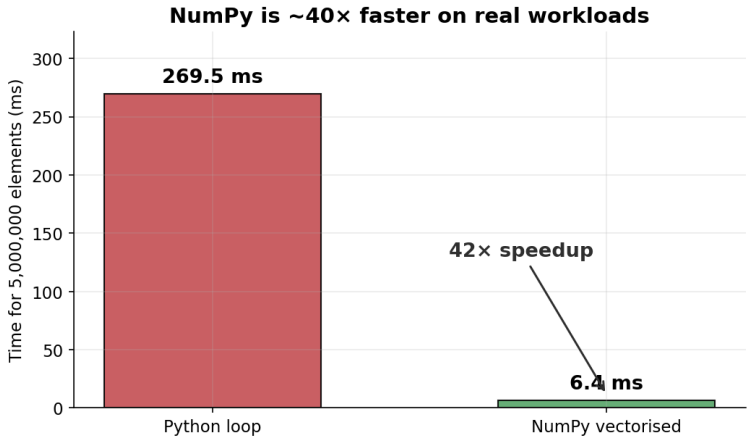
- Pandas, scikit-learn, PyTorch, TensorFlow – all NumPy underneath.
- Token counts, costs, embeddings, model weights – all live in NumPy arrays.
- Vectorised arithmetic replaces explicit loops.

The three attributes you check first on any new array

`shape`	how many rows $\times$ columns $\times \ldots$
`dtype`	what kind of number lives in each cell
`ndim`	how many dimensions

Most ML bugs are shape mismatches. `print(x.shape)` is your first debugging move, before anything else.

NumPy is ∼40× faster on real workloads



Write the formula, not the loop

```
import numpy as np

tokens_in = np.array([480, 720, 305, 610, 815])
tokens_out = np.array([120, 200, 80, 175, 240])

# One vectorised expression -- no Python loop
cost = (tokens_in / 1000 * 0.0006 +
        tokens_out / 1000 * 0.0024)

cost.sum() # total spend across the batch
cost.mean() # average per call
```

Broadcasting – the rule that powers everything

The one rule

Compare shapes **from the right**. Dimensions are compatible when they are **equal** or one of them is **1**.

Example: standardise every column of a feature matrix

```
X_std = (X - X.mean(axis=0)) / X.std(axis=0)
                        # ^^^   shape (3,) - broadcasts across all rows
```

One line replaces what would otherwise be a nested loop.

axis=0 vs axis=1 – a mnemonic

The axis that disappears

`X.sum(axis=0)` *eliminates* axis 0 (rows) → one value per column.
`X.sum(axis=1)` *eliminates* axis 1 (columns) → one value per row.

Worked example – per-column mean of a feature matrix

```
X = np.array([[1, 2, 3],          # row 0
              [4, 5, 6],          # row 1
              [7, 8, 9]])         # row 2

X.mean(axis=0)   # [4. 5. 6.]  - one mean per COLUMN  (axis 0 disappears)
X.mean(axis=1)   # [2. 5. 8.]  - one mean per ROW     (axis 1 disappears)
```

Read "axis equals k" as "this axis disappears."

Outline

1 Overview

2 Python

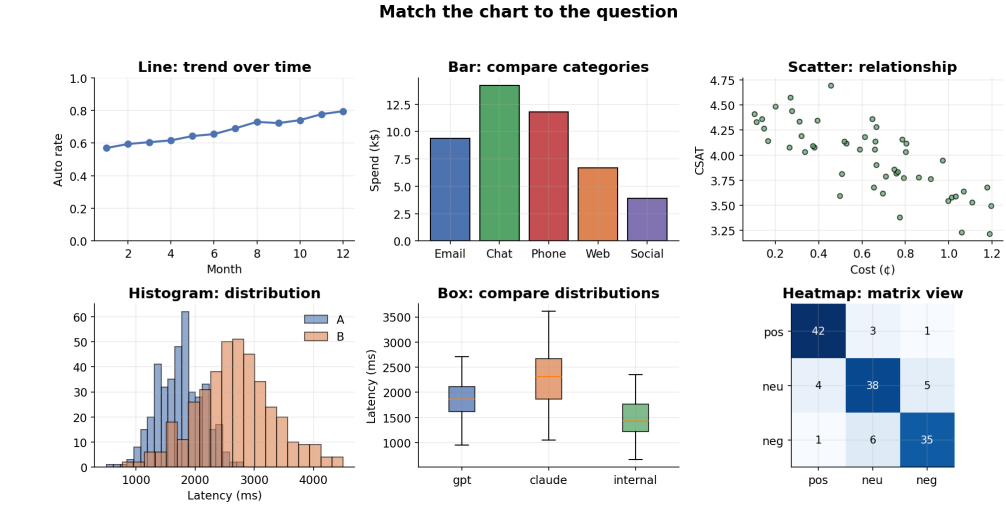
3 HTTP & SQL

4 Data Handling

5 NumPy

6 Visualisation
- Pick the right chart
- The Figure / Axes mental model

Match the chart to the question



Every chart in 7 lines

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y, color="#4C72B0", linewidth=2)
ax.set_title("Monthly automation rate")
ax.set_xlabel("Month")
ax.set_ylabel("Automation rate")
plt.tight_layout()
plt.show()
```

- `fig` = the canvas; `ax` = one plot.
- Multiple Axes per Figure → dashboards (see the capstone).
- Always label axes. A chart without context is a riddle.

Common pitfalls

The five mistakes that always show up

1. Crowded / overlapping labels → call `plt.tight_layout()`.
2. No axis labels → always set title + xlabel + ylabel.
3. Reading off the y-axis → annotate bar values directly.
4. 3-D pie charts → almost never use them.
5. Hard-to-distinguish colours → use a small consistent palette.

Outline

1 Overview

2 Python

3 HTTP & SQL

4 Data Handling

5 NumPy

6 Visualisation

7 Time Series
- Working with time-shaped data

A time series is a DataFrame with a `DatetimeIndex`

The two functions that change everything

```
df["date"] = pd.to_datetime(df["date"]) # parse strings
df = df.set_index("date") # time-aware index
```

Now you can do all of this

```
df.loc["2024-03"] # one whole month
df.resample("W").mean() # weekly aggregate
df["auto_rate"].rolling(7).mean() # 7-day smoothing
df["auto_rate"].diff(7) # week-over-week change
```

Three baselines you must beat

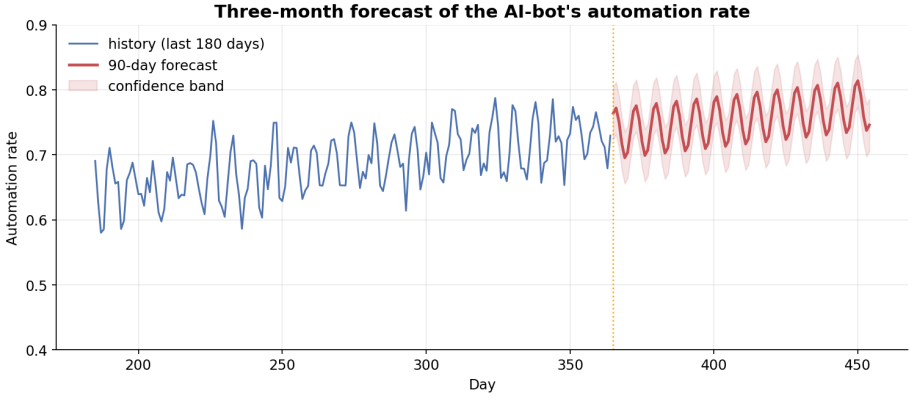
Before you reach for fancy models

1. **Naive** – tomorrow looks like today.
2. **Seasonal naive** – this Monday looks like last Monday.
3. **Moving average** – this week's average, extended.

The forecasting golden rule

Any model that doesn't beat *seasonal naive* on your data has not earned its complexity. Walk away and use the simple baseline.

A 3-month forecast in five lines



Always report a **confidence band**, evaluate with **walk-forward backtesting**, and use **MAE / RMSE / MAPE** – not a single hold-out.

Outline

1 Overview

2 Python

3 HTTP & SQL

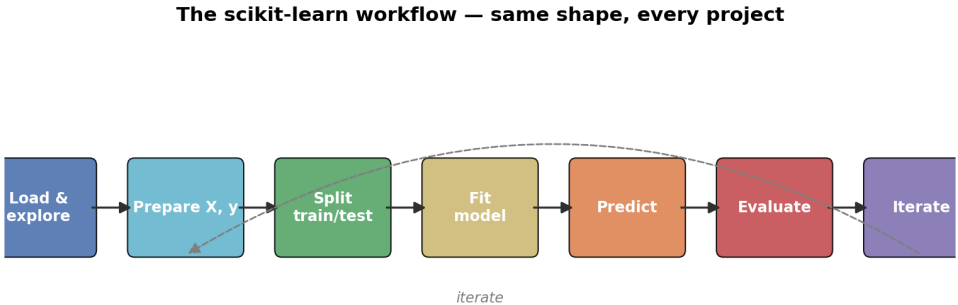
4 Data Handling

5 NumPy

6 Visualisation

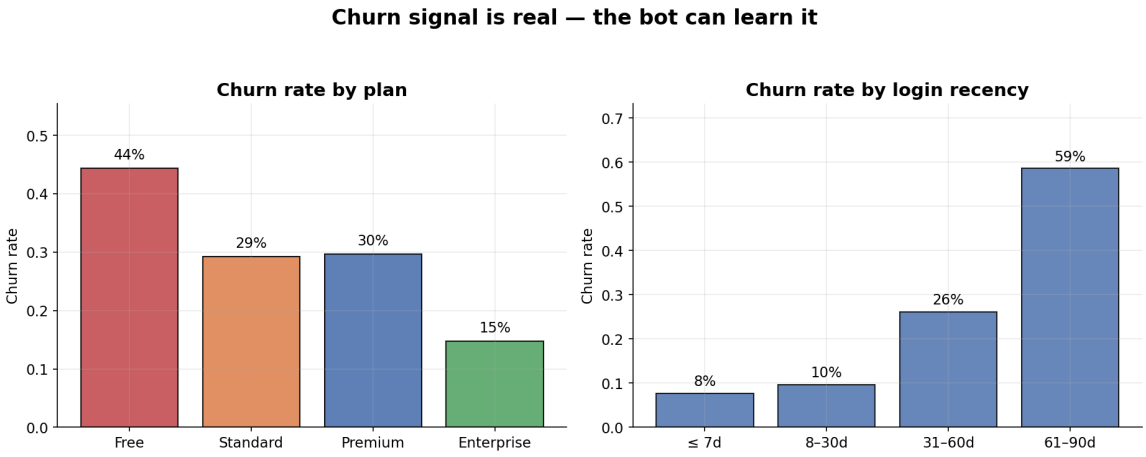
7 Time Series

The scikit-learn workflow



Every estimator follows the same `fit / predict / score` contract – swapping a logistic regression for a random forest is a one-line change.

Churn signal is real – the model can learn it



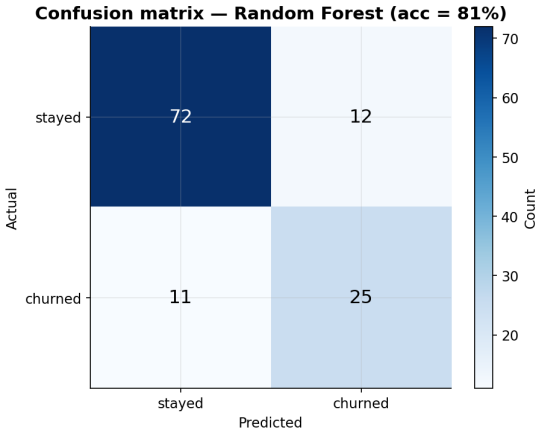
The whole project in eight lines

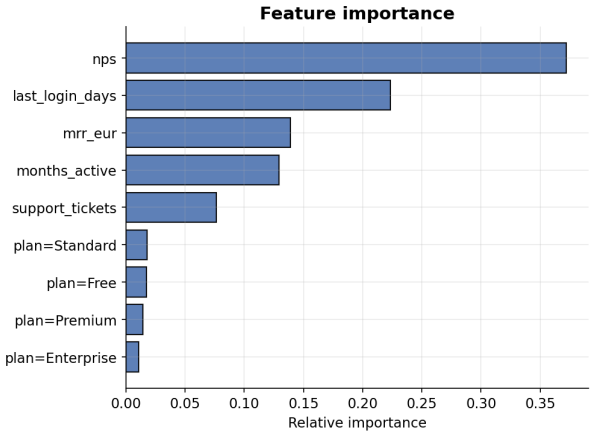
```
prep = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(), ["plan"]),
])
pipe = Pipeline([("prep", prep),
                 ("model", RandomForestClassifier(n_estimators=200))])

pipe.fit(X_train, y_train)
print(f"Accuracy: {pipe.score(X_test, y_test):.3f}") # 0.842
```

Why the Pipeline matters

The scaler and encoder are fit on *training data only* – never the test data. This single discipline prevents **data leakage**.

Model results – the diagnostics that matter





Beyond accuracy: precision, recall, F1

Which metric matters depends on cost

- **High precision** when acting on a flag is expensive (e.g. a sales follow-up).
- **High recall** when *missing* a positive is expensive (a churning customer).
- **F1** balances both – the harmonic mean.

Expose probabilities, not just labels

`predict_proba` returns $P(\text{class})$. A 51% prediction and a 95% prediction are both "positive" but tell you very different things about how to *act*.

Outline

1 Overview

2 Python

3 HTTP & SQL

4 Data Handling

5 NumPy

6 Visualisation

7 Time Series

The capstone scenario

The brief

You join a SaaS company that deployed an AI support bot at the start of the year. The product manager wants **a single document** that answers:

- Which channels is the bot working in? Which need attention?
- Is automation improving across the year?
- What's the trade-off between cost and customer satisfaction?
- Where should the team invest next?

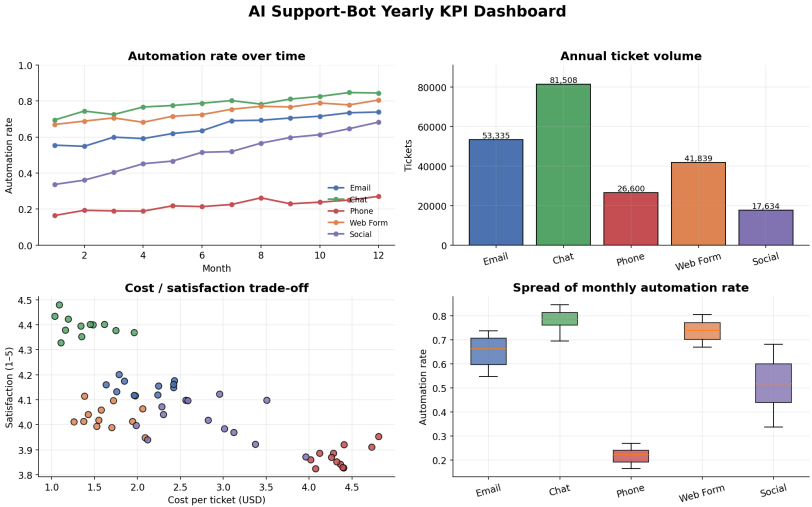
The dataset shape – 60 rows × 8 columns

Each row is one (channel, month) observation – same shape as a typical operations data-warehouse export.

Schema

Column	Meaning
`channel`, `month`	categorical – 5 channels × 12 months
`tickets_total`, `tickets_auto`	ticket counts (total / auto-resolved)
`automation_rate`	auto / total, in $[0, 1]$
`latency_ms`	median first-response latency
`satisfaction`	mean CSAT, 1–5
`cost_per_ticket`	USD per ticket (LLM + human time)

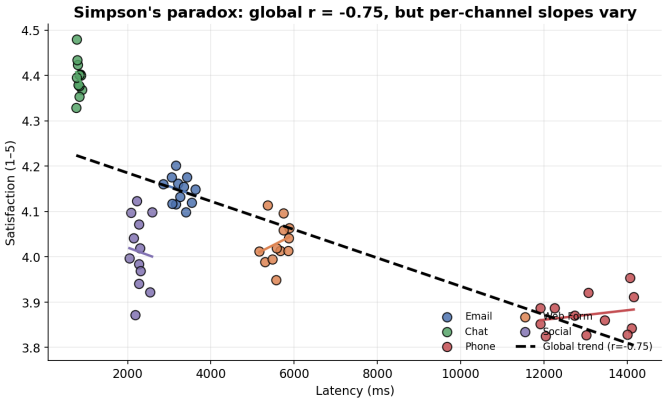
The deliverable – one slide, four panels



Reading the dashboard

- **Top-left.** Every channel trends upward – Chat and Web Form lead; Phone improves slowly from a very low base.
- **Top-right.** Volume is concentrated in Chat and Email – where automation gains have the biggest absolute payoff.
- **Bottom-left.** A clear inverse cluster: low cost / high satisfaction – the bot wins. Phone sits in the expensive zone.
- **Bottom-right.** Phone is volatile; Chat is steady. Variance matters as much as the mean.

Simpson's paradox – the trap to avoid



The lesson

The global correlation is **misleading**. Always group by category before drawing conclusions.

What you actually deliver

Five bullets a manager can read in 30 seconds

1. Overall automation rate climbed from $\sim$50% in Q1 to $\sim$75% in Q4.
2. Chat and Web Form are the bot's strongest channels and carry 50% of volume.
3. Phone is the weakest: $< 30\%$ automation, highest cost per ticket. Recommendation: route inbound voice to chat.
4. Latency does *not* drive satisfaction within a channel – the global negative correlation is a Simpson's paradox artefact.
5. Social is the breakout: double-digit uplift this year. Invest in Q1 next year.

All the code that came before exists to support these five bullets.

Outline

1. Overview
2. Python
3. HTTP & SQL
4. Data Handling
5. NumPy
6. Visualisation
7. Time Series

An LLM call is just a function call

From your code's perspective

```
response = llm(messages=[...], model="...", temperature=...)
```

What changes if you adopt this view

- Typed inputs, structured outputs, retries on failure.
- Costs you can measure, latencies you can profile.
- Tests you can run, regressions you can catch.

Treat the LLM like any other third-party API – the engineering hygiene comes free.

Three roles, one call

LLM chat anatomy — three roles, one function call

role = "system"

```
"You are a CX analyst.
Reply with one of:
 positive | neutral | negative"
```

Stable instructions →

set once, never change per request.

role = "user"

```
"I love the new dashboard!"
```

Variable input →

the actual text to analyse.

role = "assistant"

```
"positive"
```

Model response →

what your code parses next.

The four prompt patterns

Four prompt patterns that cover most AI work

Clear instructions

Role + task + constraints

```
Reply with exactly one of:
positive | neutral | negative
```

Few-shot

Show, don't tell

```
Examples:
  text → label
  text → label
  text → ?
```

Structured output (JSON)

Machine-parseable answers

```
{"sentiment": "negative",
 "topic": "billing"}
```

Chain-of-thought

Show your working

```
Step 1: identify entities…
Step 2: classify each…
Answer: …
```

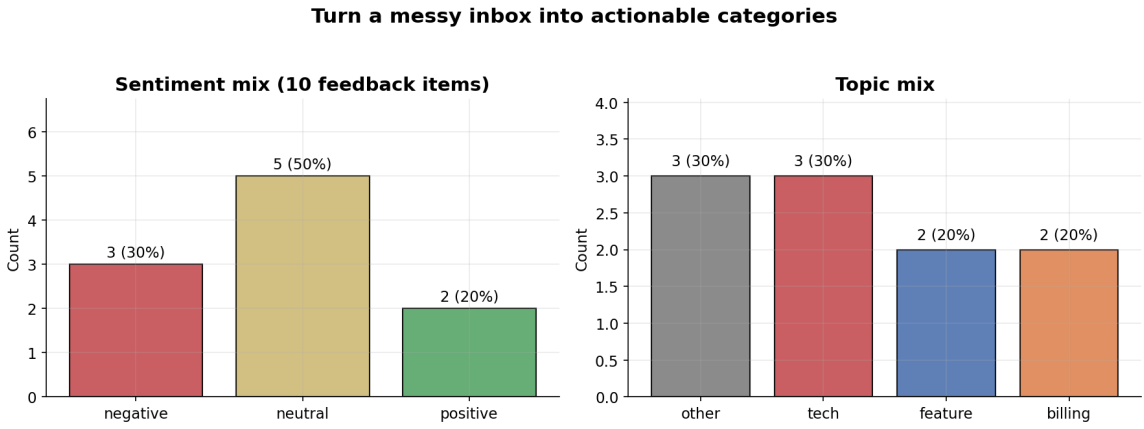
Ask for JSON – and parse it defensively

```
SYSTEM = (
    "You analyse customer messages. "
    "Return JSON with keys 'sentiment' (positive/neutral/negative) "
    "and 'topic' (billing/tech/feature/other). Reply with the JSON only."
)

resp = llm.chat(messages=[
    {"role": "system", "content": SYSTEM},
    {"role": "user", "content": text},
])

try:
    labels = json.loads(resp["text"])
except json.JSONDecodeError:
    labels = {"sentiment": "unknown", "topic": "unknown"}
```

Always wrap `json.loads` in `try/except`

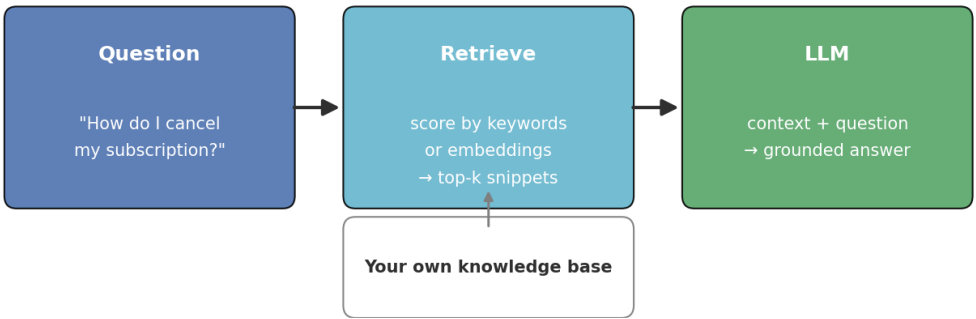
Real models occasionally add markdown fences or trailing prose.

From messy inbox to actionable categories



RAG – ground the model in your own data

Retrieval-Augmented Generation (RAG) — ground the model in your data



RAG in roughly 15 lines

```python
def rag_answer(question, docs):
    # 1. Retrieve relevant snippets
    snippets = retrieve(question, docs, k=2)
    context = "\n\n".join(f"[{name}] {text}" for name, text in snippets)

    # 2. Ask the LLM to answer ONLY from that context
    system = ("You answer the user's question using ONLY the context provided. "
              "If the context does not contain the answer, say so clearly.")
    r = llm.chat(messages=[
        {"role": "system", "content": system},
        {"role": "user", "content": f"Context:\n{context}\n\nQ: {question}"},
    ])
    return r["text"], snippets
```

Swap keyword matching for vector embeddings to upgrade to production-grade RAG.

Cost, latency, and evaluation – always

The 30-second regression test

Keep a 50–500-example labelled eval set per AI feature. Re-run it on every prompt or model change. Three columns are usually enough: `accuracy`, `latency_p95`, `cost_per_call`.

The single mistake that kills AI projects

Shipping a prompt and then *never measuring* how it behaves on real inputs. The eval set pays for itself within a week.

Swap the MockLLM for a real provider

```python
# pip install openai
from openai import OpenAI
client = OpenAI() # picks up OPENAI_API_KEY from env

def chat(messages, model="gpt-4o-mini", temperature=0.0):
    resp = client.chat.completions.create(
        model=model, messages=messages, temperature=temperature,
    )
    return {"text": resp.choices[0].message.content,
            "tokens_in": resp.usage.prompt_tokens,
            "tokens_out": resp.usage.completion_tokens}
```

Never paste API keys into a notebook

Use environment variables (`os.getenv("OPENAI_API_KEY")`) or a `.env` file plus `python-dotenv`.

Keyword RAG fails on paraphrasing

The motivating example

User asks: *"How do I end my plan?"*

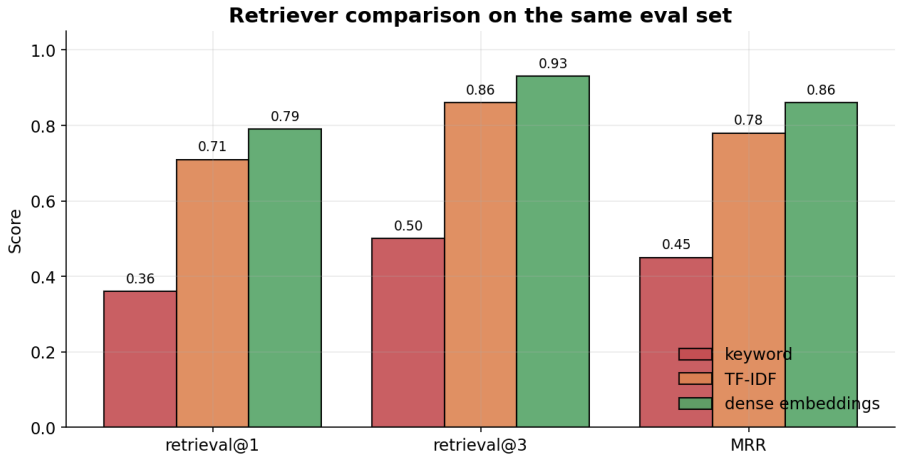
Doc says : *"Cancelling your subscription . . . "*

Zero word overlap $\Rightarrow$ keyword retrieval misses.

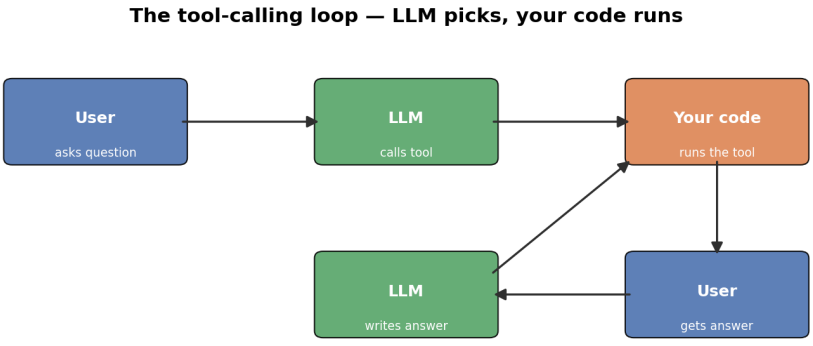
The fix: embeddings + cosine similarity

Map each text to a vector such that *similar meaning* $\Rightarrow$ *similar vector*. Distance is unit-free; synonyms work; paraphrasing works.

Retriever shoot-out on the same eval set



Tool calling – the LLM picks, your code runs



Loop until the LLM stops asking for tools.

The agent loop in five lines

```
while step < max_steps:
    resp = llm.chat(messages=messages, tools=TOOLS)
    if "text" in resp: # final answer
        return resp["text"]
    call = resp["tool_call"]
    result = TOOL_FNS[call["name"]](**call["arguments"])
    messages.append({"role": "tool", "content": json.dumps(result)})
```

Always cap the loop

`max_steps` is the single most important parameter on an agent. Without it, a buggy tool or confused model will burn through your budget silently. Set it to 5–10 for most tasks.

The pipeline is universal

Same shape for invoices, contracts, lab reports, receipts ...

`Extract` → `Chunk` → `LLM extract` → `Validate` → `Aggregate`

The output schema is your specification

```
"vendor": "Acme Cloud Solutions",
"invoice_id" : "INV-1042", "due_date" : "2024-05-15", "total" : 1802.42, "line_items" : [...]
```

Field-level accuracy is your honest metric – the worst field sets the floor.

Outline

1. Overview
2. Python
3. HTTP & SQL
4. Data Handling
5. NumPy
6. Visualisation
7. Time Series

The src/ layout – the modern Python project

The src/ layout — the modern Python project skeleton

```
costkit/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── costkit/                    → the importable package
│       ├── __init__.py              → public API (what to expose)
│       ├── cost.py                  → cost arithmetic
│       ├── parsing.py               → defensive parsers
│       └── cli.py                   → the `costkit` command
├── tests/                          → pytest, one file per module
│   ├── test_cost.py
│   └── test_parsing.py
└── scripts/                        → entry points a scheduler runs
    └── monthly_report.py
```

The three habits that pay forever

1. `pyproject.toml` declares everything – deps, version, CLI entry point.
2. `pytest` tests live in `tests/`; one file per module.
3. Install in editable mode: `pip install -e ".[dev]"` – changes to `src/` take effect immediately.

When to graduate

The moment two notebooks copy-paste the same function – or one notebook needs to run on a schedule – package it. Don't wait until later.

Pick the host that fits the job

The decision tree

- On your laptop, low frequency ⇒ **cron**
- On a Linux server ⇒ **systemd timer**
- Code already on GitHub, weekly / on-PR ⇒ **GitHub Actions**
- Multiple tasks with dependencies ⇒ **Prefect / Airflow / Dagster**

Don't pull out a heavy orchestrator until you really need it. cron handles 90% of production workloads forever.

The four production essentials

1. **Idempotency** – a deterministic `run_id` and a check before doing work.
2. **Retries with backoff** – 429 / 5xx / network blips are routine.
3. **Structured logging** – START → steps → END/FAIL.
4. **Notifications** – Slack webhook on failure; quiet on success.

The smallest production wrapper

```
def run(task_fn, run_id, max_retries=3):
    started = time.perf_counter()
    try:
        result = with_retry(lambda: task_fn(run_id=run_id),
                            max_attempts=max_retries)
    except Exception as e:
        send_slack_alert(f"FAILED {task_fn.__name__}: {e}")
        raise
    return {**result, "duration_s": time.perf_counter() - started}
```

Outline

1 Overview

2 Python

3 HTTP & SQL

4 Data Handling

5 NumPy

6 Visualisation

7 Time Series

Eight lessons that travel

1. **Functions and pipelines** beat copy-paste – every time.
2. **Defensive parsing** (`.get(key, default)`, `try/except`) is non-negotiable for external data.
3. **Vectorise** – write the formula, not the loop.
4. **Inspect before you analyse**: `head / shape / dtypes / describe / info`.
5. **Group before correlating** – Simpson's paradox is real.
6. **Test/train split, always.** Probabilities, not just labels.
7. **Match the chart to the question.** Annotate the punchline.
8. **LLMs are functions.** Structured output makes them composable.

Where to go from here

Natural extensions of this course

- Embeddings + vector stores – upgrade keyword retrieval to semantic search.
- Tool / function calling – let the LLM call your functions.
- Agents – multi-step reasoning loops that plan, act, observe.
- Evaluation frameworks – automated scoring beyond exact match.

You now have the Python fluency and the AI-engineering vocabulary to read the next tutorial and *follow what is going on* – which is more than half the battle.

Happy coding.

Open a notebook. Edit one number. Re-run. Iterate.